

AI for Economic Research: Dynamic Models, Language, and Agents

Lecture 1.2: Equilibrium Concepts & Two Classical Approaches

Zhigang Feng

Spring 2026

Topic 1.2 Agenda

1. **Equilibrium Concepts** — competitive, recursive
2. **Two Classical Approaches** — VFI, Euler equation iteration

Equilibrium Concepts

Competitive Equilibrium: Definition

- **A competitive equilibrium** consists of:

1. *Prices*: $\{r_t, w_t\}_{t=0}^{\infty}$ (rental rate and wage)
2. *Allocations*: $\{c_t, k_{t+1}, n_t\}_{t=0}^{\infty}$

such that: (i) households optimize, (ii) firms optimize, (iii) markets clear.

- **Key Equilibrium Conditions:**

- Factor prices: $r_t = \alpha A k_t^{\alpha-1} - \delta$, $w_t = (1 - \alpha) A k_t^{\alpha}$
- Household Euler equation: $u'(c_t) = \beta \mathbb{E}_t[u'(c_{t+1})(1 + r_{t+1})]$
- Resource constraint: $c_t + k_{t+1} = A k_t^{\alpha} + (1 - \delta)k_t$

- **Recursive (Markov) Equilibrium:** An equilibrium where decisions depend only on *payoff-relevant* state variables, not on calendar time or history.

- Representative agent: state = (k, z)
- Policy rules: $c = \sigma(k, z)$, $k' = g(k, z)$, prices $r = r(k, z)$, $w = w(k, z)$

From Competitive Equilibrium to Social Planner

- **First Welfare Theorem:** Under complete markets and no externalities, a competitive equilibrium is Pareto optimal.
- **Implication for Computation:**
 - For representative agent models, solve the social planner's problem
 - Recover prices from planner's marginal conditions
 - Avoids fixed-point problem over prices
- **When the Theorem Fails:**
 - Incomplete markets (Aiyagari, Bewley): no Pareto optimality
 - Externalities, taxes, monopolistic competition
 - Must solve for equilibrium prices *explicitly*
- **Practical Implication:**
 - Representative agent models: "easy" (solve planner's problem)
 - Heterogeneous agent models: "hard" (must iterate on distribution + prices)

Two Roads to Equilibrium

- We have two constructive algorithms for computing equilibrium:
- **Approach 1: Dynamic Programming (Value Function Iteration)**
 - Iterate on the Bellman equation: $v_{n+1} = Tv_n$
 - Convergence via *contraction mapping*
 - Recovers both value function and policy
- **Approach 2: Euler Equation Iteration (Greenwood-Huffman)**
 - Iterate directly on the policy function via Euler equation
 - Convergence via *monotonicity*
 - Works directly with equilibrium conditions
- Both approaches are *constructive*—they provide algorithms, not just existence proofs.

Two Classical Approaches

Dynamic Programming: The Classical Framework

- **Dynamic Programming (Bellman, 1957):** Solve sequential decision problems by exploiting recursive structure.
- **Core Idea:** Value of any state = max over (immediate reward + discounted continuation):

$$v(s) = \max_{a \in \Gamma(s)} \{r(s, a) + \beta \mathbb{E}[v(s') \mid s, a]\}$$

- **Value Function Iteration (VFI):**
 1. Discretize state space into finite grid $\{s_1, \dots, s_N\}$
 2. Initialize $v_0(s_i)$ for each grid point
 3. Iterate: $v_{n+1}(s_i) = \max_a \{r(s_i, a) + \beta \sum_j P(s_j | s_i, a) v_n(s_j)\}$
 4. Stop when $\|v_{n+1} - v_n\| < \varepsilon$
- **Requirement:** Complete knowledge of the environment— $P(s' | s, a)$ and $r(s, a)$.

Euler Equation Approach: Setup

- **Alternative:** Work directly with first-order conditions (Euler equations).
- The Euler equation for the optimal growth model:

$$u'(c_t) = \beta \mathbb{E}_t [u'(c_{t+1}) \cdot F_k(k_{t+1}, z_{t+1})]$$

- In a competitive equilibrium with $k = K$:

$$u_c(c(k, K, k', z)) = \beta \mathbb{E} [u_c(c(k', K', k'', z')) \cdot F_k(k', K', z') \mid z]$$

- **Equilibrium Requirement:** Find policy g and aggregate law of motion G such that:
 - The Euler equation holds for all (k, K, z)
 - Consistency: $G(K, z) = g(K, K, z)$ (individual = aggregate)

The Greenwood-Huffman Operator

- **Idea:** Construct a sequence of policy functions that converges to equilibrium.
- Let $H^0(K, z) = 0$. Define $H^{j+1}(K, z)$ as the x solving:

$$u_c(c(K, K, x, z)) = \beta \mathbb{E} [u_c(c(x, x, H^j(x, z'), z')) \cdot F_k(x, x, z')]$$

- **Interpretation:**

- Given a conjecture H^j for future policy, solve for today's optimal x
- The solution $x = H^{j+1}(K, z)$ becomes the new conjecture
- Iterate until convergence: $H^{j+1} \approx H^j$

- **Convergence via Monotonicity:**

- $\{H^j\}$ is monotone (increasing from $H^0 = 0$)
- Bounded by resource constraint $H^j(K, z) \leq F(K, K, z)$
- Monotone + bounded $\Rightarrow$ limit H^* exists

Comparing the Two Approaches

	Value Function Iteration	Euler Equation Iteration
Object	Value function $v(k)$	Policy function $H(K, z)$
Iteration	$v_{n+1} = Tv_n$	$H^{j+1} = TH^j$
Convergence	Contraction	Monotonicity
Theory	Banach fixed point	Monotone convergence
Strength	General, robust	Direct equilibrium
Weakness	Indirect (need to extract g)	Requires monotonicity

Both provide:

- Constructive algorithms (not just existence proofs)
- Theoretical guarantees for convergence
- Foundation for numerical implementation

Why Classical Macro Models Are “Easy”

- **Representative Agent + First Welfare Theorem:**

- Competitive equilibrium $\Leftrightarrow$ Social planner's solution
- No need to explicitly compute prices to find allocations

- **The Key Simplification:** Individual capital k = Aggregate capital K

- Prices are deterministic functions of a single state variable:

$$r(K) = \alpha AK^{\alpha-1} - \delta, \quad w(K) = (1 - \alpha)AK^{\alpha}$$

- The agent *knows* how prices evolve because $K' = g(K)$

- **By Design, Not by Accident:**

- Economists constructed this framework for tractability
- The “environment” is fully specified by $(A, \alpha, \beta, \delta)$
- State space is low-dimensional (often just k or (k, z))